

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> / *****/ /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ / *****/ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> / *****/ /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ / *****/ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> / ***** */ /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ / ***** */ </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	<pre> Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */ </pre>
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	<pre> If(record_count > limit) /* have all records been processed? */ </pre>
Bad Comment	<pre> If(record_count > limit) /* check if record count exceeds limit */ </pre>
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> / ***** */ /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ / ***** */ </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> while (miss_distance > threshold) { success_code = move_robot (target_location); if (success_code == MOVE_FAILED) { printf("The robot move has failed.\n"); } } </pre>
Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization	<pre>#define DEFAULT-NUMBER-OF-STUDENTS 15</pre>

Examples	<code>int class-size = DEFAULT-NUMBER-OF-STUDENTS;</code>
-----------------	---

Evaluation criteria and suggestions

**Evaluation
criteria**

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	To guide the development of programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /* * NombrePrograma.java * Autor: Marely Jacome * Fecha: 2025-12-10 * Versión: 1.0 * Descripción: Breve resumen del propósito del programa o clase. */ </pre>
Listing Contents	Provide a summary of the listing contents.
Contents Example	<pre> /* * Listing Contents: * - Reuse Instructions * - Declaración de atributos * - Constructores * - Métodos públicos * - Métodos privados de apoyo */ </pre>
Reuse Instructions	• Describe how the program is used. Provide the declaration format, parameter values and types, and parameter limits. • Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.

Reuse Example	<pre> /* * Reuse Instructions: * - Esta clase se utiliza para encontrar el valor de x tal que * $P(T \leq x) = p_{\text{objetivo}}$ usando la distribución t- Student. * - Declaración: * Busqueda objBusqueda = new Busqueda(); * - Llamada al método: * double x = objBusqueda.buscarX(p, dof, tolerancia, numSeg, xInicial, dInicial); * - Parámetros: * p : 0.0 < p < 0.5 (probabilidad objetivo). * dof : grados de libertad > 0. * tolerancia : valor pequeño y positivo (por ejemplo 1e-5). * numSeg : número de segmentos par (por ejemplo 10). * xInicial : valor inicial de x (por ejemplo 1.0). * dInicial : tamaño inicial del paso (por ejemplo 0.5). * - Advertencias: * * Si numSeg no es par, la regla de Simpson puede no ser correcta. * * Si dof <= 0, la función t-Student no está definida. * * Si p está fuera del rango (0, 0.5), el algoritmo puede no converger. */ </pre>
Identifiers	Use descriptive names for all variables, function names, constants, and other identifiers. Avoid abbreviations or single letter variables.

Identifier Example	Ejemplos de identificadores usados en el programa: - dblPOjetivo : probabilidad objetivo p. - intDOF : grados de libertad de la t-Student. - dblTolerancia : tolerancia permitida para el error. - intNumSeg : número de segmentos de Simpson. - dblXInicial : valor inicial de x en la búsqueda. - dblDInicial : valor inicial del paso d. - dblUltimoP : último valor de p(x) calculado. - objBusqueda : objeto de la clase Busqueda. - objSimpson : objeto de la clase SimpsonIntegration.
--------------------	--

(continued)

Coding Standard Template (continued)

Comments	• Document the code so that the reader can understand its operation. • Comments should explain both the purpose and behavior of the code. • Comment variable declarations to indicate their purpose.
Good Comment	<pre> /* Calcular el valor de p(x) usando la regla de Simpson */ dblPActual = objSimpson.integrate(intNumSeg, dblX, intDOF); /* * buscarX * Propósito: * - Dado un p objetivo, encontrar el valor de x tal que la * integral de la t-Student sea aproximadamente p. * Parámetros: * - dblPObjetivo, intDOF, dblTolerancia, intNumSeg, dblXInicial, dblDInicial. * Retorna: * - Valor de x encontrado. */ </pre>
Bad Comment	<pre> /* método */ public double buscarX(...) { ... } /* x */ double x; /* hace cosas */ doble p = objSimpson.integrate(...); </pre>
Major Sections	Precede major program sections by a block comment that describes the processing that is done in the next section

Example	<pre> /* ===== * SECCIÓN: Entrada de datos * - Leer p objetivo, dof, tolerancia y num_seg del usuario. * ===== */ ... /* ===== * SECCIÓN: Búsqueda del valor de x * - Llamar a la clase Busqueda para encontrar x. * ===== */ ... /* ===== * SECCIÓN: Salida de resultados * - Mostrar x, dof y p(x) final en pantalla. * ===== */ </pre>
Blank Spaces	• Write programs with sufficient spacing so they do not appear crowded. • Separate every program construct with at least one space.
Indenting	• Indent every level of brace from the previous one. • Open and closing braces should be on lines by themselves and aligned with each other.

Indenting Example	<pre> public void logicla() { java.util.Scanner objScanner = new java.util.Scanner(System.in); objScanner.useLocale(java.util.Locale.US); double dblPOjetivo; int intDOF; System.out.println("=== Programa 6: Búsqueda de x para t-Student ==="); System.out.print("Ingresa p objetivo (ejemplo 0.20): "); dblPOjetivo = objScanner.nextDouble(); System.out.print("Ingresa los grados de libertad (dof): "); intDOF = objScanner.nextInt(); } </pre>
Capitalization	• Capitalized all defines. • Lowercase all other identifiers and reserved words. • Messages being output to the user can be mixed-case so as to make a clean user presentation.
Capitalization Example	<pre> // Nombres de clases public class Logic { ... } public class Busqueda { ... } // Nombres de métodos public void logicla() { ... } public double buscarX(...) { ... } // Variables descriptivas double dblPOjetivo; int intDOF; double dblTolerancia; SimpsonIntegration objSimpson; </pre>